Directed Emergence of Network Structure Under an Energy Budget

Vasili Gavrilov*

2026

Abstract

A network's structure should *emerge*, not be designed, but from what, and how much of it? We grow a network's topology from an **exhaustive, fully-connected seed under an evolutionary energy budget** that pays for every connection, and ask honestly which pieces of convolution fall out. Sparse task-relevant connectivity emerges cleanly; weight sharing is *adopted* when translation invariance rewards it; and a compact aligned local filter emerges too, but only once mutation acts on the shared feature the way real genetics does, not on a single edge. On top of the priors a ConvNet hardwires (locality and translatability), the free dimensions then organize to the task: compact fields, depth that matches the required receptive field, channels that match the number of distinct operations, up to a capacity limit four interventions fail to move.

What is new: -that *directed* evolutionary search finds a *tidier* filter than random sampling at *equal task accuracy* (it is tidier, not better), which is what a directed optimizer should do on an exploitable objective and which *reconciles* the near-null our companion crossover study found on a flat benchmark landscape; -the sharing/energy *decoupling*, why a compact filter cannot emerge under per-connection pruning yet does once mutation acts on the shared feature, which restores the missing energy gradient; -the *specific evolutionary energy-emergence from an exhaustive seed*, and its honest boundary, a characterization, negatives kept, that we are not aware of a prior result showing. The rest reproduces textbook inductive-bias facts (weight sharing is data-efficient, receptive field $\approx$ depth), verified with a fair baseline rather than claimed as new. Every number regenerates from one `make`; the dead-ends are kept.

1 Introduction

This project has a long fuse. A 1997 thesis argued only that heuristics, inductive bias built into the architecture by hand, are a *necessity* for a trainable, generalizing network [11]. The ambition to instead let the architecture *emerge* from a genetic-algorithm search, not be hand-designed, in the best case rediscovering the convolutional receptive field (Figure 1), came in 2002, after the author had worked with genetic algorithms in industry; the project was created in 2015 and first implemented only now, in pure C. A companion study on that side of the idea, *searching* whole topologies and cells, reached a firmly negative conclusion: on the real NAS-Bench-101 cell space, no crossover meaningfully beats random search, and structure-aware operators do worse than a structure-blind one, because good cells are common and the gap to the optimum sits inside the benchmark's own training noise. Searching for structure did not beat random *there* (Section 2.4 returns to this under an energy budget, where the picture is different).

*Independent researcher. Physics and computer science by education; a professional software engineer with a background in numerical optimization and machine learning, including a genetic-algorithm optimizer built for industrial use in 2002. Reference implementation: `https://github.com/Anode1/SMBPANN`, 2001–2015–2026.

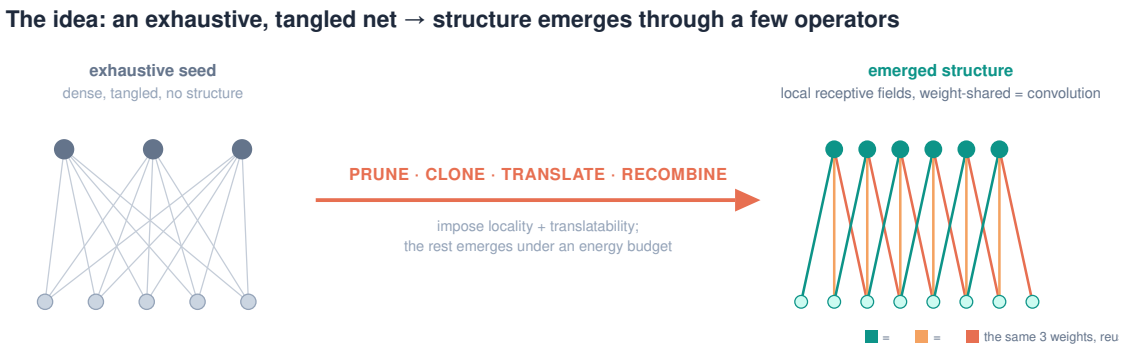

Figure 1: The idea. An exhaustive, tangled network becomes an ordered convolution through a few structural operators. We impose the domain's real symmetries (locality, translatability) and let prune, clone, translate, and recombine assemble and refine the rest.

This paper comes at the same ambition from the opposite end. Instead of searching for the architecture, we *grow* one: start from an exhaustive, fully-connected seed, put a cost on every connection, and watch which parts of convolution survive the squeeze. The question is not "can a search find a good architecture" but "what structure emerges, on its own, under an energy budget, and what refuses to."

Contribution, stated honestly. The genuinely new part is the map: the specific *evolutionary energy-emergence from an exhaustive seed*, and its boundaries. We are not aware of a prior result showing this together with the mechanism the compact filter needs in order to emerge at all (grouped mutation on the shared feature, not the single edge). This is distinct from gradient pruning (Optimal Brain Damage; the Lottery Ticket), from DARTS' gradient-relaxed supernet, and from NEAT's grow-from-minimal, which augment rather than prune. Much of the rest, weight sharing is data-efficient; receptive field $\approx$ depth, reproduces known inductive-bias facts, which we verify carefully rather than claim as new. The value is the honest characterization, the negatives, and the full reproducibility; not the underlying mechanisms.

Method in one paragraph. A one-hidden-layer network whose connectivity is a binary mask is evolved by a genetic algorithm from an all-ones (dense) seed. Fitness is validation accuracy on a scarce training set; the objective is to minimize energy (the number of connections, or free parameters) subject to accuracy $\geq$ target. Mutation is mostly-prune, rarely-grow, an annealing schedule, in which a prune is a downhill move in energy and a rare added connection is an uphill thermal fluctuation. All numbers below are means over independent seeds; each experiment is a self-contained C99 probe.

2 What emerges, and what does not

2.1 Pruning: feature selection emerges; convolution does not

Under an energy budget the search prunes a dense connectivity mask to a sparse one, and on a task whose label depends on a few specific inputs it lands **90%** of the surviving connections on the task-relevant inputs (chance 0.33, no-selection drift 0.34). Feature selection, discovering *which* inputs matter, emerges cleanly (Figure 2). What does *not* emerge is the aligned local receptive field: at comparable density the surviving connections are no more in-window than chance, so the search finds *a* sparse subnetwork, not *the* convolutional one, exactly the outcome of the pruning literature (sparse subnetworks, not convolutions).

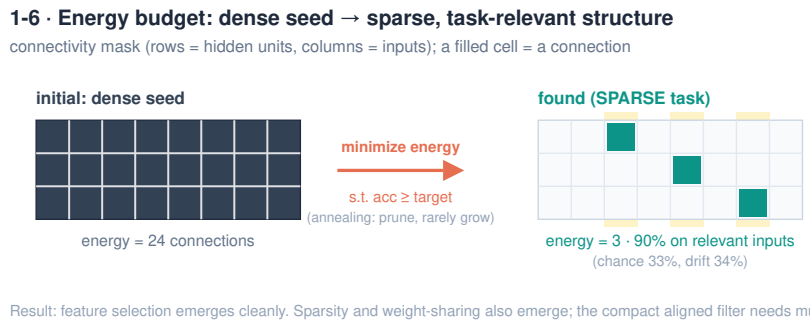

Figure 2: Sec. 2.1. An energy budget prunes a dense connectivity seed to a sparse mask whose surviving connections land on the task-relevant inputs. Feature selection and sparsity emerge; the compact aligned filter needs mutation on the shared feature to emerge (Sec. 2.3).

2.2 Imposing locality: a compact filter emerges, sharing is adopted

Locality is not something to wait out; a bounded receptive field is a law of the problem (information is local), and every ConvNet imposes it. When we impose only that, each hidden unit sees a *contiguous* window, a compact local field falls right out: the width anneals to a mean of $\approx K$ (the kernel size), the compact filter the energy budget alone could not produce. A weight-tying gene (tie weights by within-window offset, i.e. translation invariance) is *adopted* by 0.89 of the population; but its accuracy benefit is within noise, and the shared arm stays wider, because shared energy charges only the maximum width, so the mean has no gradient. The pieces of convolution keep emerging separately; the clean compact *shared* whole does not dominate.

2.3 The compact filter, and the mutation that finds it

There is a subtlety in what "an energy budget" means here, and it is, we think, the paper's one real insight. Once weights are shared, removing a single connection no longer reduces the parameter count, that offset is still used by other positions, so a mutation acting on one edge has no energy gradient to tighten the kernel. The right unit of mutation is not the edge but the shared feature.

The compact filter does emerge, under the right mutation. Biological mutation is global, not local: a change to a gene acts on every instance of the mechanism it builds (all the "similar blocks and duplicated edges"), not on one edge. So for a weight-shared filter the natural unit of mutation is the shared *offset*, all the connections that reuse that weight; flipping a whole offset removes all its duplicated edges in one move and reduces the parameter count, which restores the energy gradient a single-connection edit cannot give. Under this grouped mutation the kernel contracts: the surviving tap count falls to 2.6 (near $K = 3$) over 16 seeds, and a width penalty sharpens it to contiguity 0.80 on the generative offsets. It is not a perfect $K = 3$ kernel (span ≈ 4), and the offset genome makes the connectivity translation-invariant by construction, but the compact filter largely emerges once mutation acts on the shared mechanism, as real genetics does.

And it emerges from either direction. For completeness: run the same energy GA under grouped mutation from a *minimal* seed and grow up (a NEAT-style direction), and it reaches an essentially identical kernel to the dense seed pruned down (taps 3.5 vs 3.8, contiguity 0.77 vs 0.80, 24 seeds). The emergence does not depend on the starting point: grow-from-minimal and prune-from-dense converge on the same structure.

	paired contiguity gap (GA−random)			test accuracy	
N	fixed	scaled	denoise (r=8)	GA	random
12	+0.02	+0.02	+0.13	0.885	0.880
16	+0.06	+0.08	+0.19	0.882	0.877
20	+0.20	+0.20	+0.32	0.874	0.866
24	+0.14	+0.16	+0.19	0.866	0.862
28	+0.12	+0.17	+0.16	0.862	0.853

Table 1: Directed GA vs random at matched evaluations, 50 paired seeds (SEM ≈0.04 on each gap). The contiguity advantage is significant from N=20 and survives both a budget scaled with N and a denoising control (random given r=8 evaluations per mask, which if anything *widens* the gap). Test accuracy is tied throughout: the GA is marginally higher but always within the per-seed SD (≈0.04).

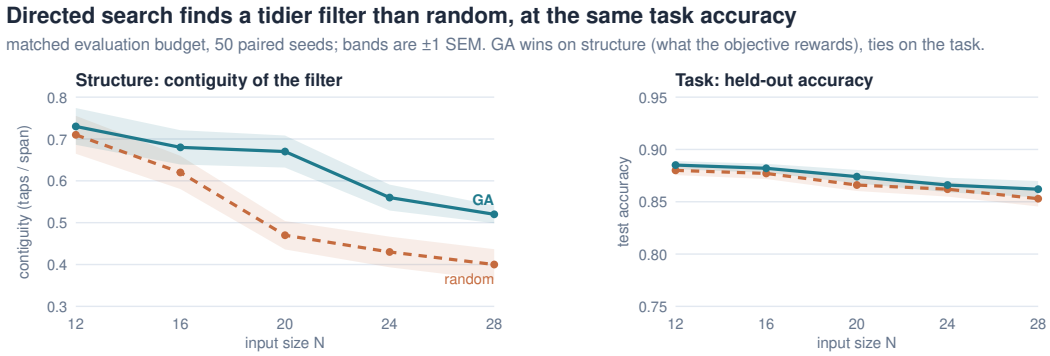

Figure 3: Directed search is *tidier, not better*. *Left:* filter contiguity, GA vs random at matched budget, the GA is clearly more contiguous. *Right:* held-out accuracy on the same seeds and the same axis honesty, the arms are tied. The GA wins on the structural term the objective rewards, not on the task. 50 paired seeds, bands ±1 SEM.

2.4 Directed search vs. random: tidier, not better

Does the *directed* energy search do real work, or would random sampling find as good a filter? The question is sharp because on a real benchmark our companion crossover study found evolution barely edges random, inside the noise. **Method.** At a *matched evaluation budget* we run the grouped-mutation GA against random offset masks (drawn across densities, kept by the same objective) over 50 *paired* seeds (same task per seed) at five input sizes N, and report both structure (contiguity) and task accuracy (Figure 3, Table 1).

Result: tidier, not better. The GA produces a markedly more contiguous filter than random from N=20 on (paired gap +0.20, +0.14, +0.12 at N=20, 24, 28, each ≥3 SEM; Figure 3 left), *but the two arms are tied on held-out accuracy at every N* (right panel; the GA is marginally higher, always within the per-seed SD). So the GA is not a better network, it is a *tidier* one at equal performance, which is exactly what a directed optimizer should do on an objective it can climb: accuracy saturates without needing a tight kernel, so the only thing left to win is the energy term, and the GA descends it by mutation where random can only sample. Two honest limits: at scale the GA is *less diffuse than random*, not compact (on-relevance falls to 0.21 at N=28, only ∼3 of 14 taps on the true offsets), and "a compact filter emerges" holds literally only at N=12.

The edge is direction, not denoising. A skeptic's confound: the GA re-evaluates each survivor with a fresh initialization every generation, so a marginal mask gets many attempts

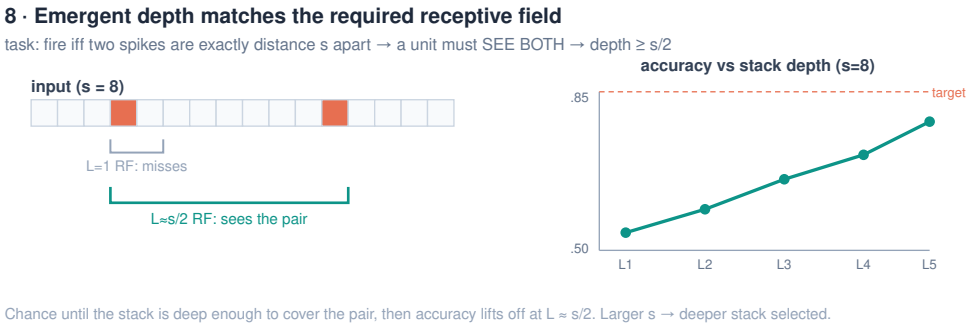

Figure 4: Emergent depth matches the required receptive field. Accuracy is at chance until the stack is deep enough for its receptive field to span the spike pair, then lifts off at depth $\approx s/2$.

at the hard accuracy gate, while random evaluates each mask once. We remove it by giving the random arm r=8 evaluations per mask (best-of-r) at the same total budget; the gap does not shrink, it *grows* (+0.20 → +0.32 at N=20), because best-of-r costs random eight-fold fewer distinct masks and exploration matters more than noise-averaging here. Scaling the budget with N firms the large-N gap slightly (+0.12 → +0.17 at N=28) but does not change the picture.

Why it matters. This *reconciles* the crossover null and confirms the project's founding conjecture, that a directed search beats random, with the condition it always quietly assumed: the objective must have structure to climb. On NAS-Bench-101 the objective was flat near the top (good cells common, differences inside training noise), so directed search had nothing to climb and tied random; here the energy objective is exploitable, so it climbs and wins. ("Tidier is really *energy-efficient*: fewer, tighter connections, the more compressed solution at equal accuracy, so the GA is the better *search*, finding a lower-energy structure than random for the same effort.) The margin should only widen with scale, where random sampling covers a vanishing fraction of a space that grows like 2^{2N} while a directed search keeps climbing, which is why large-scale architecture search uses evolution or gradients and not random; but we test $N \leq 28$, so that widening is a prediction the trend supports, not a large-net result we measure.

2.5 Emergent depth matches the required receptive field

Impose the feed-forward composition rule too, stack one reused block type, and the one dimension left free, the depth, emerges to fit the task. On a task that fires only when two spikes are exactly distance s apart (so a unit must see both at once, needing receptive field $\geq s{+}1$, i.e. depth $\geq s/2$), accuracy sits at chance until the stack is deep enough to span the pair, then lifts off at depth $\approx s/2$; larger s selects a deeper stack (Figure 4). This is the clearest positive of the emergence side: given the necessary priors, the free dimension organizes itself to the task.

3 The operators: reuse, recombination, translation

3.1 Reuse beats a free search at equal compute

Composing blocks explodes combinatorially, so we adopt the heuristic LeCun did: reuse one block type, stacked, and search only the depth. At *equal compute*, a free heterogeneous search (a GA over arbitrary block sequences, using a dilated-conv library) never solves more reliably or at lower energy than simply enumerating uniform reused blocks; on the harder tasks reuse is strictly better. Block diversity buys nothing but a larger search, the reuse heuristic is the tractable near-optimum, which is the user's intuition ("the combinations explode, so reuse the same block") made quantitative. This mirrors cell-based NAS (ENAS, DARTS), which search one cell and stack it.

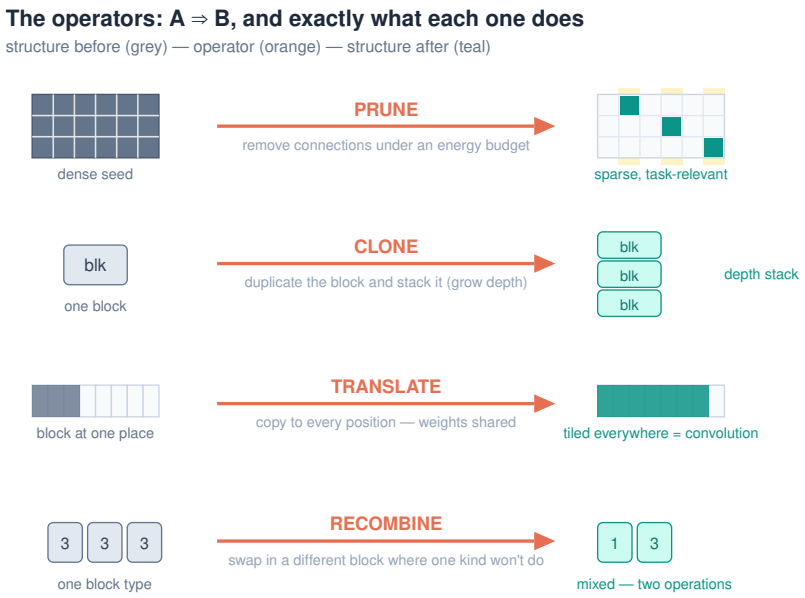

Figure 5: The four structural operators as $A \to B$, each with its exact action. Prune (remove connections under an energy budget), clone (duplicate and stack, for depth), translate (copy to every position with shared weights, a convolution), recombine (swap in a different block where one kind will not do). Each maps to a biological analog: pruning, gene/segment duplication, transposition/serial homology, sexual recombination.

3.2 Recombination is required when a task needs two different operations

Reuse breaks exactly where the biology predicts. On a task that needs a fine local motif $(+, -, +)$ at adjacent positions (visible only to a dilation-1 block) *and* a far spike at distance s (reached cheaply only by a dilation-3 block), no single reused block serves both roles: a uniform dilation-3 stack is blind to the fine motif (solving $0/8$), a uniform dilation-1 stack cannot afford the reach. Recombination, a GA over mixed block sequences, solves $8/8$ at about half the energy, and its solutions are literal two-operation mixes such as $[1, 3]$: one fine block, one coarse-reach block (Figure 6). Clone when scaling a working module; recombine only when you need a new kind of part, runners when the structure repeats, seeds when it must change.

3.3 Translation: weight sharing is data-efficient (a reproduction, fairly done)

The third operator, translation, replicating one working detector across positions with shared weights (transposition, serial homology, cortical-map tiling), is exactly convolution's tiling, reached by replication. On a translation-invariant motif task we compare a weight-shared filter (3 weights) against a locally-connected baseline (one filter per position). Table 2 gives the fair result (mean over restarts, ± 1 SD over 40 seeds): the shared filter wins at every train size, most so when data is scarcest (gap $+0.24$ at 16 examples, $+0.14$ at 128 as the baseline slowly catches up). This is the textbook argument for weight sharing (one filter learns the motif from every position's examples while each independent filter starves), here quantified with a fair estimator. It is a phenomenon of weight *sharing*, not of architecture search: the two arms are weight-tied vs untied, both trained by plain SGD.

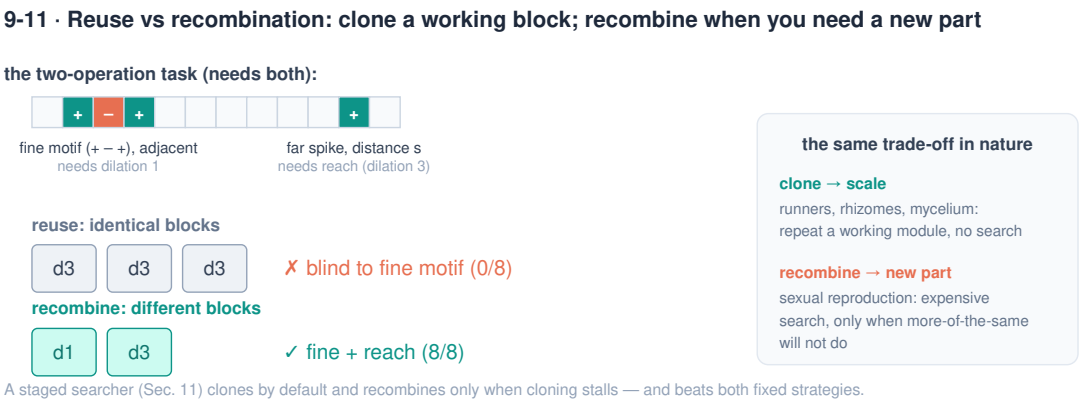

Figure 6: Reuse vs recombination, and the clone/recombine trade-off nature also makes. One reused block suffices for repetitive structure; a task needing two different operations forces recombination of different blocks.

train size	weight-shared	locally-connected	gap
16	0.820 ± 0.122	0.581 ± 0.065	$+0.239$
32	0.838 ± 0.094	0.615 ± 0.071	$+0.224$
64	0.866 ± 0.082	0.672 ± 0.076	$+0.195$
128	0.867 ± 0.081	0.724 ± 0.086	$+0.143$

Table 2: Weight sharing is data-efficient. Shared filter (3 weights) vs locally-connected (42 weights), by training-set size. The advantage is largest at the scarcest data.

3.4 The advantage widens with input size, then saturates

Sweeping input size N (so the number of positions to cover grows), the shared arm stays flat at ≈ 0.94 (always 3 weights, size-independent) while the locally-connected baseline falls to chance (0.52) and bloats to 378 weights, because each per-position filter starves on the $\approx n_{\text{tr}}/P$ examples that land there (Figure 7, 250 seeds). The gap is 4–6× the SD, so the effect is significant. Honestly stated it *widens then saturates*: most growth is $N{=}16 \to 32$, after which the baseline has already hit the chance floor. The effect survives even a fully-trained, oracle-supervised locally-connected baseline (which we hand the motif locations): the shared arm still wins $+0.14$ at $N{=}16$ up to $+0.38$ at $N{=}128$, so it is not an artifact of the baseline's training rule.

4 An honest boundary in 2-D

In two dimensions "two different operations" means two feature *channels*, a horizontal and a vertical filter, since one channel is one orientation. A task requiring both orientations present caps a single channel near 0.75 (the analytic ceiling a lone orientation detector can reach), and accuracy rises with channels ($0.69 \to 0.77 \to 0.86 \to 0.89$ over $C{=}1\ldots4$, 24 seeds): a gradual rise, not a sharp $C{=}2$ jump, but the load-bearing fact, one channel is not enough for two operations, is significant. The tidy generalization "channel count = operation count," however, **fails**: growing channels to solve a K-operation task gives selected $C^* \approx 1.2, 3.2, 5.0$ for $K{=}1, 2, 3$, so $K{=}2$ already overshoots and $K{=}3$ breaks, sub-target at every channel count ($C{=}5$: 0.79), only 2 of 24 seeds solving.

This K=3 wall is genuine, not under-training. Four interventions fail to move it: more channels, a nonlinear (MLP) readout, competition (lateral inhibition, the mechanism that self-

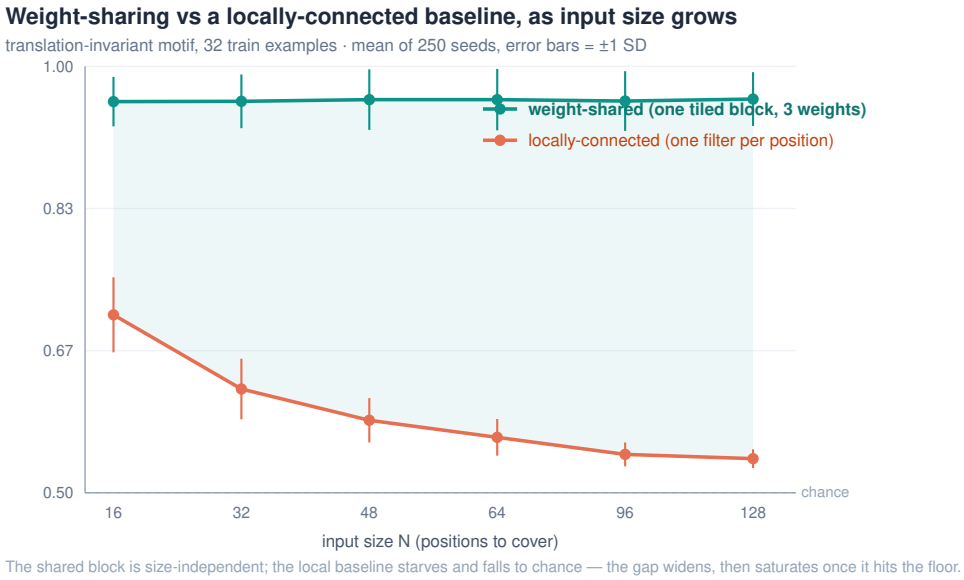

Figure 7: The weight-sharing advantage as input size grows (mean of 250 seeds, error bars ±1 SD). The shared block is size-independent; the locally-connected baseline starves and falls to chance. The gap widens, then saturates once the baseline hits the floor.

organizes cortical orientation columns), and, with a budget caveat, a deeper extractor. An independent heavy-compute rerun (2.5× epochs, 2× signal) leaves K=3 sub-target at every channel count, confirming the wall is structural for the channel axis. Per-channel detection error compounds multiplicatively ($0.95^3 \approx 0.86$, at the target edge), and no cheap refinement moves a multiplicative wall; real vision clears such tasks with scale (depth, normalization, residuals, vast data), not one clever pressure. We report this boundary plainly: it is more honest than stopping at small K where the staircase still looked clean.

5 Related work, and how we differ

The closest neighbors, and the honest distinction: **DARTS** starts from an exhaustive supernet and prunes to a final architecture, closest in spirit, but via continuous gradient relaxation over predefined operations, not an energy GA, and it never asks whether convolution emerges. **Optimal Brain Damage** and the **Lottery Ticket Hypothesis** prune dense to sparse, but by gradient magnitude, and find sparse subnetworks, not convolutions. **NEAT**, **HyperNEAT**, and Cellular Encoding evolve topology but *grow from a minimal seed*, the opposite direction to pruning from exhaustive. **Cell-based NAS** (ENAS, regularized evolution) reuses one cell and stacks it, which our reuse observations echo. We claim the honest characterization of what an evolutionary energy budget does and does not produce, and the negatives, not the underlying mechanisms, which are known.

Two framings place this experiment in a longer line. The view of network topology as problem-specific inductive bias that must be engineered by hand goes back decades [11]; here we ask the complementary question, whether that structure can instead *emerge* from the energy budget rather than be built in.[1] Viewed more broadly, the energy budget is a description-length pressure: the emergent weight-shared filter is a compressor whose inductive bias, locality and sharing, matches the task's translation symmetry, an instance of the evolutionary-search route to compressor discovery set out in [10].

[1]The broader claim that such emergence is substrate-independent is argued informally in a companion essay [12].

6 Limitations

The tasks are small synthetic constructions built so the intended structure is optimal; they demonstrate what emerges under an energy budget on a task shaped like convolution, and do not by themselves generalize to real data. The reuse/scale results are weight-*sharing* data efficiency, not architecture search; we relabel them accordingly. The full developmental pipeline chaining all four operators on a genuinely compositional task, and a real dataset, are the natural next steps; the K=3 "wall" is a capacity limit of *this* shallow conv + max-pool setup, and whether a properly trained deeper net clears it is open. The honest home of this work is an explainer or a negative-results workshop, not a novelty venue.

7 Conclusion

Impose the domain's real symmetries; let a few biological operators assemble the rest. Under an energy budget on an exhaustive seed, sparsity and feature selection emerge and weight sharing is adopted, and a compact convolution emerges too, but only once mutation acts on the shared feature rather than the single edge, for a decoupling reason we can state in a line. On top of imposed priors, depth emerges to match the receptive field, reuse beats a free search at equal compute, recombination is required only when a task needs a genuinely different operation, and weight-shared tiling is data-efficient at a degree that grows with scale then saturates. Where it runs out, a three-way conjunction unmoved by four fixes, we say so. The contribution is the honest map, the negatives, and the reproducibility; every number regenerates from one command.

References

[1] Y. LeCun, J. S. Denker, S. A. Solla. Optimal Brain Damage. *NeurIPS* 1990.

[2] Y. LeCun et al. Backpropagation Applied to Handwritten Zip Code Recognition. *Neural Computation* 1(4), 1989.

[3] J. Frankle, M. Carbin. The Lottery Ticket Hypothesis. *ICLR* 2019.

[4] H. Liu, K. Simonyan, Y. Yang. DARTS: Differentiable Architecture Search. *ICLR* 2019.

[5] H. Pham, M. Guan, B. Zoph, Q. Le, J. Dean. Efficient Neural Architecture Search via Parameter Sharing. *ICML* 2018.

[6] K. O. Stanley, R. Miikkulainen. Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation* 10(2), 2002.

[7] E. Real, A. Aggarwal, Y. Huang, Q. V. Le. Regularized Evolution for Image Classifier Architecture Search. *AAAI* 2019.

[8] S. Kirkpatrick, C. D. Gelatt, M. P. Vecchi. Optimization by Simulated Annealing. *Science* 220, 1983.

[9] V. Gavrilov. SMBPANN reference implementation, 2001–2015–2026. `https://github.com/Anode1/SMBPANN`.

[10] V. Gavrilov. Intelligence Is the Discovery of Compressors. 2026. `https://doi.org/10.5281/zenodo.20440111`.

[11] V. Gavrilov. Backpropagation Feed-Forward Neural Networks. Seminar thesis, Open University of Israel, 1997 (digitized 2026). `https://doi.org/10.5281/zenodo.20450526`.

[12] V. Gavrilov. Emergence Does Not Care About Substrate. Companion essay, 2026. https://gavr144.substack.com/p/emergence-does-not-care-about-substrate.